Business Value of GenAI & AI Costs.

Table of contents.

What you'll be able to do.

By the end of this module you'll be able to:

01 Pick the right metric for a GenAI use case in your team.

02 Draft a one-page business case a sponsor will actually read.

03 Estimate token, GPU and hidden costs before approval.

04 Reason about cloud vs open-source vs on-prem with real numbers.

05 Apply caching and right-sizing to cut run cost 50–75%.

Why this matters.

What will this be worth to the bank?

If you can't answer in one sentence with one number, the project will not be funded — or worse, funded once and never renewed.

FOUR WAYS PROJECTS FAIL WITHOUT A CLEAR NUMBER

DEMO TRAP	Pilot looks great in a demo — quietly dropped after six months because nobody measured anything.
PHANTOM SAVINGS	Value claimed in “hours saved” — but those hours were never re-deployed to revenue work.
COST BLOWOUT	Run cost in production turns out to be 5× the pilot — sponsor pulls funding.
RISK REVIEW	Compliance asks “what's the benefit?” during model risk review — team has no answer.

The four metrics that get funded.

Pick one — secondary metrics are evidence

01

Time Saved

Hours / FTE / week. Cut from a process that used to take humans.

02

Error Reduction

% defects ↓. Fewer mistakes that trigger rework or losses.

03

Cost / Transaction

₼ per unit ↓. Lower unit cost on a high-volume process.

04

Revenue Uplift

₼ new or retained. More sold, better conversion or retention.

A simple framework for ROI.

ROI = (Annual benefit − Annual cost) / Total cost to deliver

Healthy pilot target: ROI > 100% within 18 months · Payback < 12 months

Five inputs you need

01	02	03	04	05
Volume	**Unit benefit**	**Adoption**	**Run cost**	**Build cost**
Units of work / year	₼ saved or earned per unit	% users/cases the AI covers	Tokens + infra + ops / year	One-off integration + training

Worked example · call-centre summariser.

Inputs

Volume: 2.0M calls / year
Unit benefit: 90 sec saved/call
× ₼1.04 fully-loaded agent cost/min
= ₼1.56 / call
Adoption: 70% of eligible calls
Run cost: ~₼0.09 / call (model + infra)
Build cost: ₼805k one-off (integration, change)

Calculation

Annual benefit
= 2.0M × 70% × ₼1.56
= ₼2,184,000

Annual run cost
= 2.0M × 70% × ₼0.09 = ₼126,000

Net annual benefit = ₼2,058,000

ROI (yr 1) = (2,058 − 805) / 805 = 156%
Payback ≈ 4.7 months

How leading banks measure AI value.

Five categories that show up consistently

01	**Time / productivity**	Hours saved/FTE, % work AI-assisted, code-suggestion acceptance rate
02	**Quality / risk**	Defect rates, complaints, model-risk findings, human override rates
03	**Unit economics**	Cost-to-serve, cost-per-document, cost-per-decision — quarterly
04	**Revenue**	Cross-sell uplift, lead-to-meeting conversion, save-desk retention
05	**Adoption & trust**	Weekly active users, repeat usage, NPS, escalation-to-human rate

Anatomy of a one-page business case.

If it doesn't fit on one page, it won't get read

01

Problem

One sentence. Whose problem, how big, what's broken today.

02

Solution

What the AI does — at the level a sponsor understands.

03

Primary metric

The ONE number that will go up or down. With a target.

04

Cost & timeline

Build cost, year-1 run cost, time-to-value.

05

Risks & controls

Top 3 risks (model, data, compliance) and mitigations.

06

Ask

Funding, sponsor, decision needed by when.

Exercise 1 · draft a business case.

Groups of 3–4 · 30 minutes

01 Pick a real workflow from someone's team in your group

02 Fill in the one-page template (problem → ask)

03 Choose ONE primary metric and put a target on it

04 Estimate volume, unit benefit, adoption — order of magnitude is fine

05 Identify the single biggest risk and how you'd manage it

06 Be ready to pitch in 2 minutes

TIPS

1. Real workflow > clever idea
2. One metric, not four
3. Round numbers — refine later
4. If you can't name the sponsor, the case isn't ready

What is a token?

The unit AI providers actually bill you for

01 A token is a chunk of text — roughly 3–4 characters in English

02 1 token ≈ ¾ word · 1,000 tokens ≈ ~750 words

03 Models read tokens in (input) and write tokens out (output)

04 Prices quoted per million tokens — input usually cheaper than output

05 Reasoning models also charge for “thinking” tokens you never see

EXAMPLE

“Transfer £500 to account 12345.”

→ tokenises to roughly:

8 tokens for a short sentence.

How tokens drive cost.

Three levers

01

Input tokens

Everything you send the model: user question, system instructions, documents, conversation history.

02

Output tokens

Everything the model writes: answer, reasoning shown, structured fields. Typically 3–6× input price.

03

Model choice

Bigger models cost more per token. A premium model can cost 10–30× a small one for the same task.

Cloud pricing models.

Three flavours you'll meet in vendor conversations

01

Pay-per-token

Per-million in/out. No commitment, scales with usage.

Good: pilots, variable workloads.
Watch: bill shocks at scale.

02

Subscription / seats

Flat per-user/month for a packaged product.

Good: predictable team rollouts.
Watch: paying for inactive seats.

03

Enterprise licence

Negotiated capacity, dedicated throughput, residency.

Good: scale, compliance, BYO-data.
Watch: minimums, lock-in.

Cloud models · real pricing.

Per 1M tokens · approximate list prices · verify on provider page

Provider	Model tier	Input ฿/M	Output ฿/M	Context	Banking fit
Anthropic	Claude · flagship	฿26	฿128	200K	Reasoning, long-doc analysis
Anthropic	Claude · mid	฿5	฿26	200K–1M	Workhorse — most use cases
Anthropic	Claude · small	฿1.70	฿8.50	200K	High-volume, latency-sensitive
OpenAI	GPT · flagship + o-series	฿9–26	฿26–102	128K–200K	Reasoning, code, agentic
OpenAI	GPT · small	฿0.26	฿1.02	128K	Cheap classification / extraction
Google	Gemini · Pro	฿2.13	฿8.50	1M+	Very long context, multimodal
Google	Gemini · Flash	฿0.13	฿0.51	1M	Cheapest long-context
DeepSeek	V3 / R1	฿0.46	฿1.87	128K	Frontier-class, very cheap
Mistral	Large / Small	฿3.40	฿10.20	128K	EU residency options

Prices move quarterly. Output tokens cost 3–6× input. Reasoning tokens billed as output.

Open-source models.

“Open weights” — download and run on your own hardware (or a 3rd-party host)

Origin	Model family	Sizes	Notes for banking
Meta	**Llama 3.3 / Llama 4**	8B – 405B	Industry standard. Vast ecosystem of tools & fine-tunes.
Mistral	**Mistral Large / Small**	7B – Large	European vendor. Strong on EU languages, EU residency options.
DeepSeek	**DeepSeek V3 / R1**	671B MoE	Frontier-class. Very cheap inference. Data-residency review.
Alibaba	**Qwen 2.5 / 3**	0.5B – 72B	Strong multilingual. Wide size range. Residency caveats.
Google	**Gemma 2 / 3**	2B – 27B	Small, easy to self-host. Good for edge / on-device.
Microsoft	**Phi-3 / Phi-4**	3.8B – 14B	Compact yet strong reasoning. Easy to fine-tune.

Three ways to consume open models

1. Hosted by the model vendor · 2. 3rd-party inference provider (Together / Fireworks / Groq) · 3. Self-hosted on your GPUs

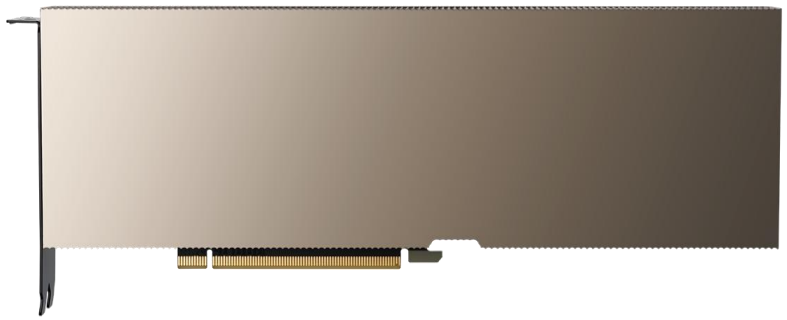
GPUs for AI · NVIDIA family.

Data-centre, workstation and personal AI cards · what's already running at Pasha Technology

GPU	Class	Memory	Notes	Pasha Technology status
B200 (Blackwell)	Data centre	192 GB HBM3e	Frontier training & inference.	—
H200	Data centre	141 GB HBM3e	Long-context inference, large models.	Coming June
H100	Data centre	80 GB HBM3	Enterprise inference workhorse.	—
A100 80GB	Data centre	80 GB HBM2e	Mature, plentiful, lower cost.	—
RTX 6000 Ada	Workstation	48 GB GDDR6	Pro workstation card — dev & on-prem inference.	Coming June
L40S	Inference	48 GB GDDR6	Cost-effective small/mid-model inference.	Deployed
RTX 4090	Consumer	24 GB GDDR6X	Consumer card — dev & prototyping.	2× deployed
DGX Spark	Personal AI	128 GB unified	NVIDIA GB10 superchip personal AI workstation.	4× deployed

Pasha Technology Deployed: 2× RTX 4090 · 4× DGX Spark · L40S · Coming June: RTX 6000 Ada · H200

On-prem GPU economics.



NVIDIA A100 · Wikimedia Commons

01 Hardware — GPUs dominate (~60% capex)

02 Power & cooling — 15–25% of hardware / year

03 Maintenance & support — 8–15% / year

04 Staff — 4–6 platform team (MLOps, security)

05 Refresh — GPU generations move every 18–24 months

Illustrative 3-year TCO · 8× H100 cluster

Hardware	Power, cooling	Support	Platform team	3-yr total
₼580–690k	**₼90–140k/yr**	**₼60–90k/yr**	**₼345–460k/yr**	**₼1.8–2.5M**

Cloud vs on-prem.

No universal answer — but there are clear signals

Cloud / hosted API wins when…

01 Variable, bursty usage

02 You need the latest models — no refresh cycle

03 Data can leave with the right controls

04 Speed-to-value > per-token cost

05 You don't yet know steady state

On-prem / private wins when…

01 Volume is huge and steady (millions/day)

02 Data residency / sensitivity blocks external

03 Latency tolerance is tight and predictable

04 You have the MLOps team to run it safely

05 TCO beats cloud at your scale

Hidden costs · integration & monitoring.

These almost never appear in the original pitch deck

Integration

01 Data sources — auth, schemas, freshness

02 UI / embedding into existing tools

03 Identity & access — who can call, with what data

04 Audit trails & lineage — required by model risk

05 Change management — training, support, comms

Monitoring

01 Quality evals after model updates

02 Drift — inputs & outputs changing over time

03 Cost telemetry — who spent what, where

04 Safety / abuse — prompt injection, sensitive content

05 Incident response — what happens at 2am

Cost lever 3 · caching, three types.

“Don't pay twice for the same answer” — each works differently

01

Prompt caching

PROVIDER SIDE

Provider caches model state for repeated prefix.

Next call pays a fraction of input price.

· Anthropic ~90% off
· Google ~75% off
· OpenAI ~50% off

02

Response caching

YOUR SIDE

You store request → response pairs.

Identical request = instant, zero model cost.

· FAQs / canned answers
· Repeated lookups
· Read-mostly tools

03

Semantic caching

YOUR SIDE

Embed query, look up vector-similar past queries.

Catches paraphrased repeats exact-match misses.

Watch: false matches, stale answers.

Prompt caching · how it works.

Structure your prompt so the stable parts come FIRST

01	**System prompt**	Same on every call	**CACHED — pay ~10–50% of normal**
02	**Attached docs**	Policy / product PDFs	**CACHED**
03	**Conversation hist**	Recent turns	**Cacheable in chunks**
04	**User question**	Varies every call	**FULL PRICE — but small**

TTL ~5 min idle · minimum cacheable size ~1k tokens · output quality unchanged — pure price arbitrage

Caching in practice · banking patterns.

Where each cache type earns its keep

01	**Policy / compliance Q&A**	**Prompt cache**	60–85% off on the policy portion of every call.
02	**Internal FAQ assistant**	**Response cache**	Near-zero marginal cost on hits. TTL = your review cycle.
03	**RM client-summary bot**	**Prompt cache**	Cache the taxonomy. Client record stays uncached — fine.
04	**Search / triage chat**	**Semantic cache**	“My card got locked” ≈ “how do I unlock my card?”
05	**Code assistant**	**Prompt cache**	Repo context dominates token count — fastest payback.

Prompt caching · anatomy of a request.

What the provider sees, and what it can safely reuse

REQUEST

```
system_role:
    "You are a compliance assistant. Use the"
    "policy below to answer staff questions."

policy_document:    [~30,000 tokens, stable]
    <internal AML / KYC policy file>

few_shot_examples:  [~2,000 tokens, stable]
    <three worked Q&A examples>

# ———————  CACHE  MARKER  ———————

user_question:      [~200 tokens, varies]
    "Can a corporate client open an account"
    "remotely without an in-person notary?"
```

EVERYTHING ABOVE THE MARKER

→ Cached prefix

On repeat calls, provider re-uses the stored state. You pay ~10% of input price on this part.

EVERYTHING BELOW THE MARKER

→ Full price

Changes on every call, so cannot be reused. Keep this part SHORT to maximise savings.

Cache lifecycle: write · hit · expire.

What happens on call 1, calls 2…N, and after the TTL

T = 0 | T = 0 → +5 min | T = +5 min idle

1 Call 1 — write

CACHE WRITE

Provider computes the internal state for the stable prefix.

Stores it for the next ~5 min.

Charge: ~1.25× input price

Cost vs uncached **+25%**

2 Calls 2 … N — hit

CACHE HIT

Provider matches the prefix byte-for-byte, re-uses the stored state.

Charge: ~10% of input price

Cost vs uncached **−90%**

3 After ~5 min idle

CACHE EXPIRE

TTL elapsed. Cache dropped.

Next call writes a fresh one.

Keep traffic warm — or use extended-TTL markers where

Effect **Back to full**

Worked example · policy Q&A bot.

1,000 questions per hour against a 30,000-token policy doc

SETUP

Stable prefix	**30,000 tokens**
User question	**~200 tokens**
Model output	**~500 tokens**
Traffic	**1,000 calls / hr**
Input price	**₼5.10 / 1M tokens**
Output price	**₼25.50 / 1M tokens**
Cached input	**₼0.51 / 1M (90% off)**
Cache write	**₼6.38 / 1M (+25%)**

Net saving: ~82% on this workload

WITHOUT CACHING

```
Per call:
  input   30,200 × ₼5.10/M  = ₼0.154
  output     500 × ₼25.50/M = ₼0.013
  ──────────────────────────  ────────
  total per call                ₼0.167

Per hour (1,000 calls):
```

₼167 / hr

WITH CACHING

```
~12 writes / hr (5-min TTL):
  30,000 × ₼6.38/M  = ₼0.192

~988 hits / hr:
  30,000 × ₼0.51/M  = ₼0.015
    +200 × ₼5.10/M  = ₼0.001
    +500 × ₼25.50/M = ₼0.013
  ──────────────────  ──────
  per hit               ₼0.029
```

₼31 / hr

Prompt structure for max cache hits.

Put stable content FIRST, variable content LAST

01	Identity / role	Same on every call	CACHED
02	Task instructions	Same on every call	CACHED
03	Policy / documents	Same for a long time	CACHED
04	Few-shot examples	Same per use case	CACHED
▼ CACHE MARKER ▼			
05	Conversation history	Changes between turns	PARTIAL
06	Current user question	Changes every call	FULL PRICE

RULES OF THUMB

- **Stable first** — Put the biggest unchanging chunk (policy, system prompt) at the very top.
- **Variable last** — Anything that changes between calls goes AFTER the cache marker.
- **Min size** — Cache typically needs ≥ ~1,024 tokens of stable prefix to kick in.
- **Byte-exact** — A trailing space or different comma = cache miss. Treat the prefix as immutable.
- **Watch the TTL** — ~5 min idle by default. Keep traffic warm or use longer-TTL markers.
- **Cache invalidation** — If the cached doc changes (policy update), the cache is stale — refresh it.

Context · what the model sees.

The model's working memory for ONE call — sized in tokens, capped per model, billed every time

WHAT IS CONTEXT?

Everything the model sees on a single call. Re-loaded on EVERY call — no memory between requests.

system · documents · history · tool outputs · user question

Token budget = the size of this whole bar. Bills every token, every call.

CONTEXT WINDOWS — CURRENT FRONTIER

Origin	Model	Window	Equivalent
Google	Gemini 2 Pro	2 M	~1,500 pages
Google	Gemini 2 Flash	1 M	~750 pages
Anthropic	Claude (1M tier)	1 M	~750 pages
Anthropic	Claude standard	200 K	~150 pages
OpenAI	GPT-4.5 / o3	200 K	~150 pages
OpenAI	GPT-4o	128 K	~96 pages
DeepSeek	V3 / R1	128 K	~96 pages
Meta	Llama 3.3 / 4	128 K	~96 pages

WHAT CHANGES AS CONTEXT GROWS

! **Cost rises linearly**	Every extra token = more $ on every single call. Pricing is per-token.
! **Latency rises**	More tokens to process = slower first-token and total response time.
! **Lost in the middle**	Models recall the start and end better than the middle of long context.
! **Distraction risk**	Irrelevant content dilutes attention — quality can DROP with more context.

HOW TO MANAGE CONTEXT

✓ **Use RAG, not stuffing**	Retrieve only the relevant chunks; don't paste 100-page documents.
✓ **Summarise history**	Compress old turns into a 1-paragraph summary, drop raw text.
✓ **Stable content first**	Put system + docs at the top so they cache; user input last.
✓ **Key facts at start AND end**	Counter the lost-in-the-middle effect — repeat critical instructions.

Banking takeaway: bigger window ≠ better — only the part you actually use is useful. Treat context like a budget, not unlimited storage.

Putting the levers together.

How a real cost-optimisation programme plays out

01	**Week 1**	**Measure**	Turn on per-call cost telemetry. Find top 3 use cases by spend.
02	**Week 2**	**Trim**	Audit prompts. Cut context, cap outputs, add JSON. Re-measure.
03	**Week 3**	**Route**	Tier the model. Send easy traffic to a smaller model.
04	**Week 4**	**Cache**	Prompt caching for stable context, response/semantic for FAQs.
05	**Ongoing**	**Govern**	Budgets per use case. Alert on overruns. Review quarterly.

Realistic outcome: 50–75% reduction in run cost within a month, no quality regression.

Exercise 2 · cost a real use case.

Same groups · 25 minutes

01 Estimate input tokens per call (rough: words × 1.3)

02 Estimate output tokens per call

03 Multiply by calls/year × adoption → annual token volume

04 Pick a price tier (small / mid / large) — use the table

05 Add hidden costs: integration, monitoring, compliance

06 Identify your biggest cost-saving lever

Deliverable

One number on the back of an envelope:

· Year-1 run cost (£)
· Build cost (£)
· Where you'd attack first to bring run cost down

Be ready to share in 90 seconds.

Common budgeting traps.

Pin these to the wall before the next vendor meeting

01	**“Our pilot cost £X — production will too”**	Production usually 3–10× pilot run cost.
02	**“The vendor's calculator says…”**	Calculators ignore context bloat & compliance.
03	**“We'll just use the cheapest model”**	Cheap + retries can cost more than mid + one good answer.
04	**“It's free, bundled in our licence”**	Bundled usually means ‘capped’ — find the cap.
05	**“We'll figure out monitoring later”**	‘Later’ = after the first incident. Build day one.

What “good” looks like.

Five things to take back to your team

01 Every GenAI use case has ONE primary metric, with a target and a date

02 Build cost, run cost, and hidden cost are on the same page as the benefit

03 Per-call cost telemetry is on from day one — you can't optimise what you can't see

04 Prompts are reviewed like code — context trimmed, outputs capped, caching on

05 Models & hardware are tiered — flagship reserved for calls that need it

Recap · what we covered.

A map of the last 3–4 hours

Part A · Business Value

01 Four metrics: time, errors, cost, revenue

02 Five-input ROI framework

03 How leading banks measure value

04 One-page business case template

05 Exercise: drafted real cases

Part B · AI Costs

01 Tokens & real model pricing (cloud + open)

02 GPU options: NVIDIA, AMD, TPU, Trainium, Gaudi

03 Hidden costs: integration, monitoring, compliance

04 Three levers: trim, right-size, cache (×3 types)

05 Exercise: costed real cases

Glossary · quick reference.

Token
A chunk of text the model reads or writes (~¾ word).

Inference
Running the model in production to produce an answer.

Prompt cache
Provider-side reuse of a stable prefix — cheaper repeats.

MoE
Mixture-of-experts — only some weights activate per token.

Hallucination
A confident, fluent, wrong answer.

Context window
Total tokens a model can consider at once.

HBM
High-bandwidth memory on the GPU — sets max model size.

RAG
Retrieval-augmented generation — fetch data, then ask.

Quantisation
FP16 → INT8/INT4 — fits smaller / cheaper GPUs.

Guardrail
A check around the model — block, redact, escalate.

GenAI Bootcamp 26.05.2026
Sessiya İştirakı